



ΠΑΝΕΠΙΣΤΗΜΙΟ
ΠΑΤΡΩΝ
UNIVERSITY OF PATRAS

Department of Electrical & Computer Engineering

Robotic Systems I (ECE_DK808)

Project 5: Biped Locomotion Challenge - Simulation, Walking Control

Kalloniatis Ilias Marios
Student ID: 1100879
up1100879@ac.upatras.gr

Kanellopoulos Konstantinos
Student ID: 1100881
up1100881@ac.upatras.gr

Academic Year: 2025–26

Demonstration videos & source code

The robot videos (flat / incline / stairs walking, push recovery, omnidirectional walking, and the extended demonstrations) together with the full source code are available at:

<https://github.com/mk4ll/Biped-Locomotion-Challenge>

Abstract

We present a dynamics-first, torque-level walking controller for two humanoid robots - the **Unitree G1** (33 kg, 29 DoF) and the **PAL Talos** (94 kg, 32 DoF) - in the MuJoCo simulator. At every control step a *whole-body Quadratic Program (QP) inverse-dynamics* problem is solved and *joint torques* are commanded, rather than tracking kinematic references with position servos. The controller is organised as a clean offline $\rightarrow$ online $\rightarrow$ whole-body-control pipeline: an offline layer plans footsteps and a Zero-Moment-Point reference; an online layer closes the loop on the *Divergent Component of Motion* (DCM / capture point) and reactively adjusts foot placement; and the QP realises the resulting Centre-of-Mass, swing-foot, torso and posture tasks subject to the floating-base dynamics, surface-frame friction cones and torque limits. Expressing all balance quantities with respect to gravity and the local contact surface lets the same controller generalise from flat ground to inclines and small stairs without redesign. The system achieves continuous flat walking, single-support balancing, weight shifting, push recovery, omnidirectional walking (forward / backward / strafing / turning), uphill walking up to 16° , and stair climbing; a simple slip-versus-tip analysis predicts the static standing limit ($\approx 27^\circ$, i.e. $\arctan \mu$), which the experiments confirm. The robot-agnostic design runs every task on both robots. We additionally showcase two integrated demonstrations of path planning and disturbance handling: a “waiter” that weaves a slalom around randomly generated tables while keeping a tray level, and a “Sisyphus” that pushes a boulder up a slope.

Contents

1	Introduction	3
2	Theoretical Background	3
2.1	Floating-base dynamics	3
2.2	Whole-body control as a QP	3
2.3	LIPM, ZMP and the Divergent Component of Motion (DCM)	4
2.4	Friction cones and the incline	4
3	System Architecture	4
4	Implementation	4
4.1	Robot models and torque actuators	5
4.2	Dynamics terms	5
4.3	Whole-body QP and tasks	5
4.4	Planning: footsteps, FSM, DCM, swing	5
4.5	Terrain-aware control (flat / incline / stairs)	6
5	Results: Walking and Balance	6
5.1	Gravity compensation (dynamics sanity check)	6
5.2	Standing balance: weight shift and single support	7
5.3	Continuous flat walking	8
5.4	Inclines and stairs	9
5.5	Omnidirectional walking	11
5.6	Push recovery	11
5.7	Second robot: PAL Talos	12
6	Incline Limit Analysis	13
7	Extended Demonstrations	13
7.1	Waiter: slalom navigation around tables	14
7.2	Sisyphus: pushing a boulder up a slope	14
8	Parameters	15
9	Discussion, Limitations and Future Work	15
10	Conclusion	16

1 Introduction

Bipedal locomotion is a canonical underactuated control problem: a humanoid is supported only through unilateral, friction-limited contacts and must continuously move its centre of mass (CoM) over the supporting foot to step without falling. This project (*Biped Locomotion Challenge*) targets a *simulation-only* walking controller for a humanoid in MuJoCo, focusing on balance maintenance, weight shifting, and stable double-support $\leftrightarrow$ single-support transitions.

Objectives (assignment core tasks).

- (a) generate stable, continuous walking;
- (b) plan and control CoM trajectories for stable locomotion;
- (c) shift the body weight to stabilise over the support foot in single support;
- (d) coordinate foot placement and body motion for continuous stepping;
- (e) maintain stability through support-phase transitions;
- (f) recover from small disturbances and imperfect foot placement.

Approach and contributions. We adopt a *dynamics-first* design: a per-step whole-body QP inverse-dynamics controller that outputs joint torques, driven by a DCM-based planning/feedback layer. Our main contributions are:

- a torque-level whole-body QP with hard floating-base dynamics and *surface-frame* friction cones, with a safe fallback when infeasible;
- a DCM (capture-point) planning and feedback layer with reactive step adjustment for push recovery;
- a *terrain-aware* formulation that generalises to inclines and small stairs, plus a short slip-vs-tip analysis of the standing limit;
- a *robot-agnostic* stack that runs identically on the G1 and Talos;
- omnidirectional walking and two integrated path-planning demonstrations.

2 Theoretical Background

2.1 Floating-base dynamics

With generalised coordinates q and velocities v (the first 6 DoF being the unactuated floating base), the rigid-body dynamics in contact are

$$M(q)\dot{v} + h(q, v) = S^\top \tau + \sum_c J_c^\top f_c, \quad (1)$$

where M is the mass matrix, h the bias (Coriolis, centrifugal and gravity) term, S the actuation selection matrix, and J_c, f_c the contact Jacobian and force at each contact point.

2.2 Whole-body control as a QP

Tasks are imposed at the acceleration level, $a_{\text{des}} = a_{\text{ref}} + K_p e + K_d \dot{e}$, giving residuals $J_i \dot{v} - b_i$. With decision variables $x = [\dot{v}; \tau; f]$ we solve

$$\min_x \sum_i w_i \|J_i \dot{v} - b_i\|^2 \text{ s.t. Eq. (1), } J_c \dot{v} + \dot{J}_c v = 0, \text{ friction cones, } \tau_{\min} \leq \tau \leq \tau_{\max}. \quad (2)$$

2.3 LIPM, ZMP and the Divergent Component of Motion (DCM)

Under the Linear Inverted Pendulum Model the Zero-Moment Point satisfies $p_{\text{zmp}} = x_{\text{com}} - (z/g) \ddot{x}_{\text{com}}$. Defining the DCM (capture point) $\xi = x + \dot{x}/\omega$ with $\omega = \sqrt{g/z}$ splits the CoM dynamics into a stable part and the single unstable mode

$$\dot{\xi} = \omega (\xi - p_{\text{zmp}}), \quad (3)$$

which yields a clean feedback law and makes “where to step not to fall” $\approx$ “step toward the DCM”.

2.4 Friction cones and the incline

Each contact force must lie in its linearised friction cone, expressed in the *surface* frame (normal n = surface normal, not world- z): $f_n \geq f_{\min}$, $|f_{t1}| \leq \mu f_n$, $|f_{t2}| \leq \mu f_n$. On a slope of angle α , a static body *slips* when $\tan \alpha > \mu$ and *tips* when the gravity line leaves the support; we use this in Sec. 6.

3 System Architecture

The controller is a three-stage pipeline: an **offline** layer (footstep planner, ZMP reference, backward-recursion DCM trajectory, swing-foot splines, gait FSM); an **online** layer (DCM feedback, capture-point step adjustment, terrain-aware friction orientation, CoM-height regulation, heading); and the **whole-body QP** (2) that outputs torques to MuJoCo, with state feedback closing the loop.

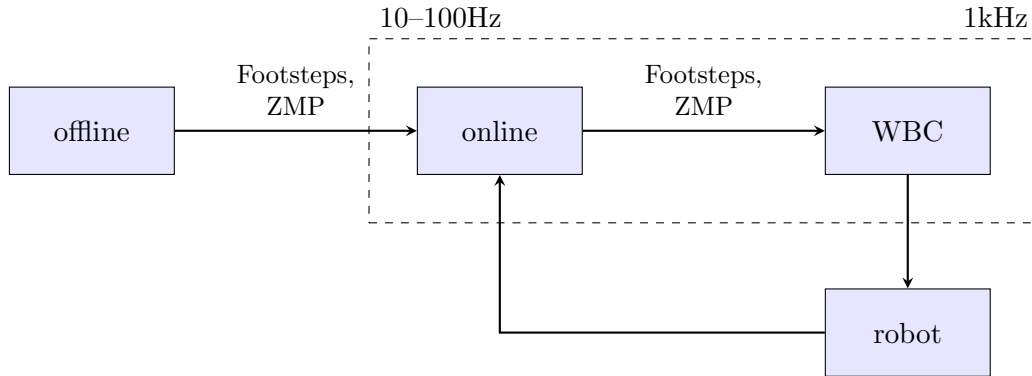


Figure 1: System architecture and control loops

4 Implementation

The controller is implemented in Python on MuJoCo 3.4. The code is organised by the pipeline of Sec. 3: `src/sim` (model loading and a thin environment wrapper), `src/dynamics` (model terms and contacts), `src/control` (the whole-body QP, tasks, and the walking controller), and `src/planning` (footsteps, FSM, DCM, swing trajectories, terrain). A single `config/params.yaml` holds every gain, weight and timing; nothing is hard-coded. The QP is solved with `quadprog` (a dense, strictly convex solver) and falls back to `clarabel/osqp` if a step is infeasible.

4.1 Robot models and torque actuators

Both robots are loaded from the MuJoCo Menagerie. The G1 ships with *position* servos, so at load time we rewrite all 29 actuators to *motor* (torque) actuators and re-apply each joint’s torque limit (e.g. ± 88 N m at the hips, ± 139 N m at the knees), since motors do not inherit `actuatorfrange`. We add four contact-corner sites under each foot (used as the contact points for the friction cones). The PAL Talos (`scene_motor.xml`) already ships torque motors but uses *box* feet, so we inject the four corner sites onto the foot box via `mjSpec` at load time. A single `RobotConfig` (`src/sim/robots.py`) captures everything that differs between the two robots — base/torso bodies, foot sites and corners, the bent-knee crouch pose, and the ankle joints — making the entire controller *robot-agnostic*: the same code runs the 33 kg G1 and the 94 kg Talos. Both start from a statically balanced *bent-knee crouch* (feet flat, CoM lowered to ≈ 0.65 m for the G1) so that walking shows naturally flexed knees; the arms are parked in a fixed posture and held by the posture task.

4.2 Dynamics terms

Per control step, `ModelTerms` extracts the quantities of Eq. (1): the mass matrix M via `mj_fullM`, the bias h via `data.qfrc.bias` (which already contains Coriolis, centrifugal and gravity terms), the selection matrix S , the translational contact Jacobians of the foot corners (`mj_jacSite`), and a 6-D Jacobian at each foot site for the rigid contact constraint. The convective term $\dot{J}v$ is obtained from the spatial site acceleration with $\ddot{q} = 0$; crucially we *also* zero gravity while computing it, because MuJoCo’s `mj_objectAcceleration` otherwise folds the gravitational offset into the acceleration and corrupts $\dot{J}v$ — a subtle bug that, before being fixed, made the QP believe the robot was in free fall.

4.3 Whole-body QP and tasks

The QP (2) uses decision variables $x = [\dot{v}; \tau; f]$. The floating-base dynamics (1) enter as a hard equality. A rigid foot would make the four corner constraints redundant (12 scalar equations for a 6-DoF foot), which the solver flags as infeasible; we therefore impose the *non-acceleration* constraint once, as a *6-D constraint at the foot site* (full rank), while keeping a 3-D force per corner for the friction pyramid and centre-of-pressure. Friction cones are built in the *surface frame* via a rotation R_{surf} (identity on flat ground, the slope rotation on an incline), and torque limits enter as bounds. The cost stacks the weighted task residuals (Table 1): a high-weight **CoM** task, a **torso/pelvis orientation** task that keeps the trunk vertical *with respect to gravity*, a low-weight **posture** regularisation toward the crouch, and a **swing-foot** task that is active only while a foot is airborne. The CoM command itself is not a plain position PD: in the horizontal plane it is the DCM feedback law derived from (3) ($\dot{\xi}_{\text{cmd}} = \dot{\xi}_{\text{ref}} + k(\xi_{\text{ref}} - \xi)$), then $p_{\text{zmp}}^{\text{cmd}} = \xi - \dot{\xi}_{\text{cmd}}/\omega$ and $a_{\text{com}} = \omega^2(x - p_{\text{zmp}}^{\text{cmd}})$, while the vertical axis tracks a low-passed CoM height. Joint torques are read directly from the QP solution and sent to MuJoCo.

4.4 Planning: footsteps, FSM, DCM, swing

The offline layer (`src/planning`) produces a continuous reference. The **footstep planner** places alternating left/right steps from a desired step length (or a body-frame velocity command, or an arbitrary 2-D path), with a kinematic clamp. The **gait FSM** builds the phase timeline $\text{DS}_{\text{init}} \rightarrow \text{SS} \rightarrow \text{DS} \rightarrow \dots$ with the support/swing assignment and the per-phase ZMP endpoints. The **DCM planner** fixes the ZMP at the stance foot during single support and integrates (3) *backward* (the stable direction) from a terminal condition $\xi_{\text{end}} = p_{\text{zmp, end}}$, then integrates the CoM *forward*; this keeps the ZMP inside the support polygon by construction. The **swing planner** uses a smoothstep in the horizontal plane and a $(1 - \cos)$ vertical bump (zero foot velocity at lift-off, apex and touchdown for a soft landing). An online **capture-point**

adjustment shifts the swing-foot target by $k(\xi_{\text{meas}} - \xi_{\text{ref}})$, updated throughout the step, which is what provides push recovery (Sec. 5). The planner outputs are sampled by the controller as $\text{reference}(t)$ and are completely independent of the robot — Fig. 2 shows the offline plan alone, before any robot moves.

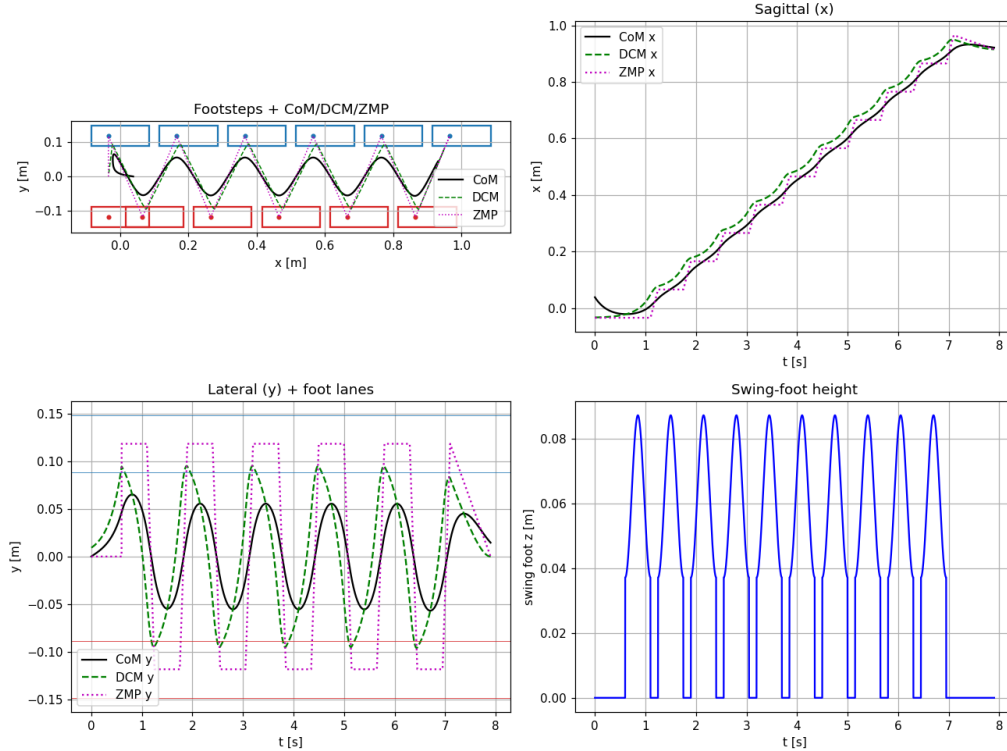


Figure 2: Offline planning output (no robot yet): footstep placement with the CoM, DCM and ZMP trajectories, and the swing-foot height. The ZMP steps from foot to foot, the DCM (capture point) leads the CoM, and the CoM weaves smoothly between the feet.

4.5 Terrain-aware control (flat / incline / stairs)

Terrain awareness is what lets the *same* controller walk on flat ground, inclines and small stairs. A `Terrain` object (`src/planning/terrain.py`) does two jobs: it mutates the model at load time via `mjSpec` (tilting the floor for an incline, adding stair boxes), and it exposes `height(x, y)`, `normal(x, y)` and a per-terrain `footstep_x`. Footsteps then land *on* the surface ($z = \text{height}(x)$), the swing foot is oriented to the surface normal so it lands flat, the friction cones are rotated to the surface (R_{surf} from `normal`), and the CoM-height reference is regulated relative to the local ground and low-passed (which removes the vertical jerk at each stair edge). On stairs the `footstep_x` snaps each foot onto a tread centre so the sole never overhangs a riser.

5 Results: Walking and Balance

All results are headless and reproducible (a single `evaluate.py` battery reproduces them); figures are produced by the corresponding scripts. Unless noted, numbers are for the G1.

5.1 Gravity compensation (dynamics sanity check)

Before any walking, we verify that the dynamics terms are correct: we solve the static balance $S^T \tau + \sum J_c^T f = h$ in double support and command the resulting feedforward torques. The

robot stays upright and essentially still (base height drift ≈ 1.7 mm, dynamics residual $\sim 10^{-4}$), proving that M , h and the contact Jacobians are consistent (Fig. 3).

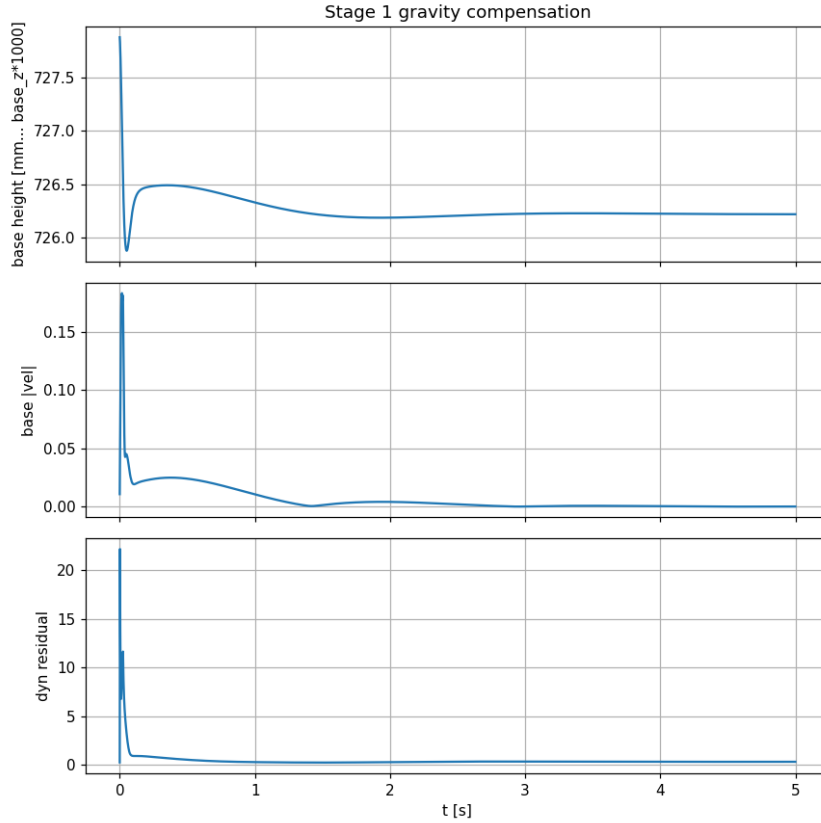


Figure 3: Gravity compensation: the robot holds its pose by feedforward torque alone (base height, base velocity and dynamics residual vs. time).

5.2 Standing balance: weight shift and single support

Driven by the whole-body QP, the robot stands stably, shifts its CoM laterally ($\pm$ several cm), and balances on a single foot while the other is raised (Fig. 4). The QP is feasible at every step.

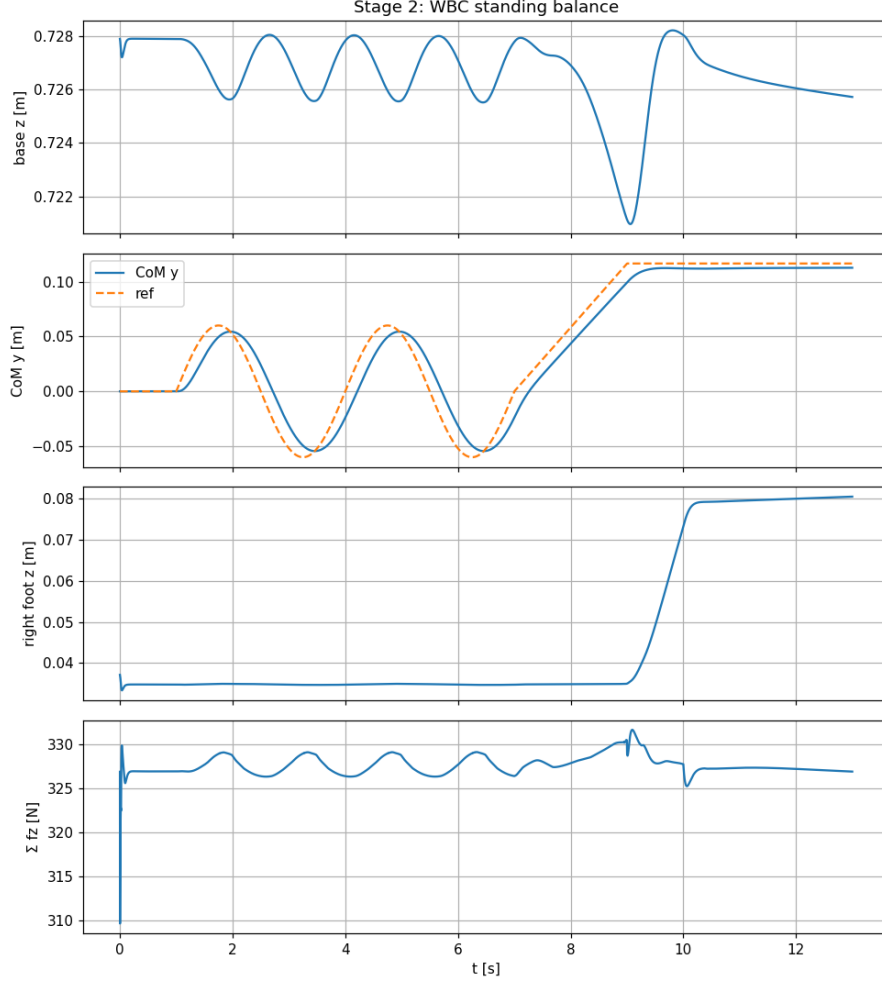


Figure 4: Standing balance: base height, lateral CoM tracking during a weight shift, swing-foot height during single support, and total normal force.

5.3 Continuous flat walking

With the DCM feedback and capture-point step adjustment, the G1 walks a continuous, stable gait for the full plan (~ 10 steps, ~ 1 m) without falling, with small CoM tracking error and a pelvis tilt of only a few degrees (Fig. 5).

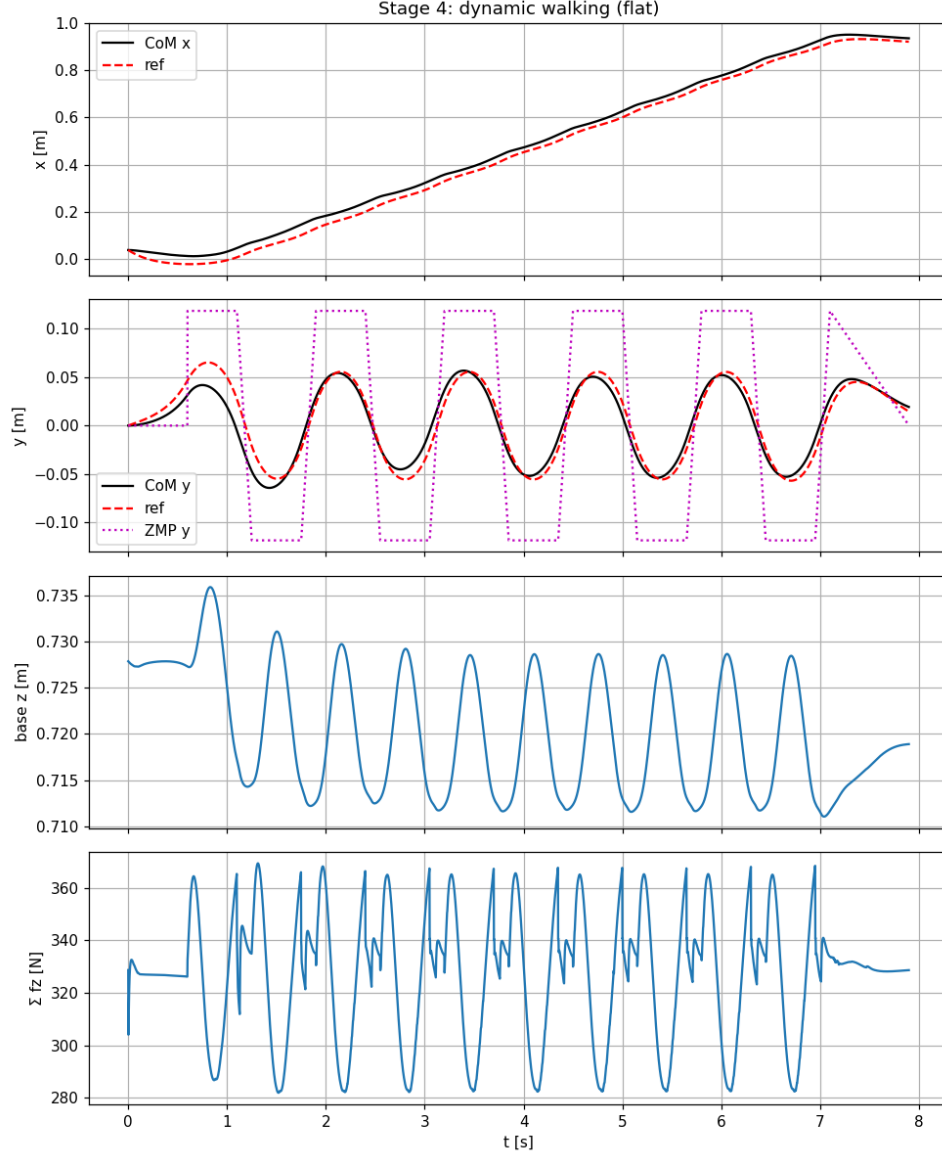


Figure 5: Continuous flat walking: forward and lateral CoM vs. the reference, base height, and total contact force (the alternating SS/DS load transfer).

5.4 Inclines and stairs

Because footsteps, friction cones and CoM height are all expressed relative to the local surface, the same controller walks uphill and climbs small stairs. The G1 walks up inclines up to 16° (Fig. 6) and climbs a full staircase tread-by-tread (Fig. 7); panel (B) of each figure shows the CoM height rising with the terrain.

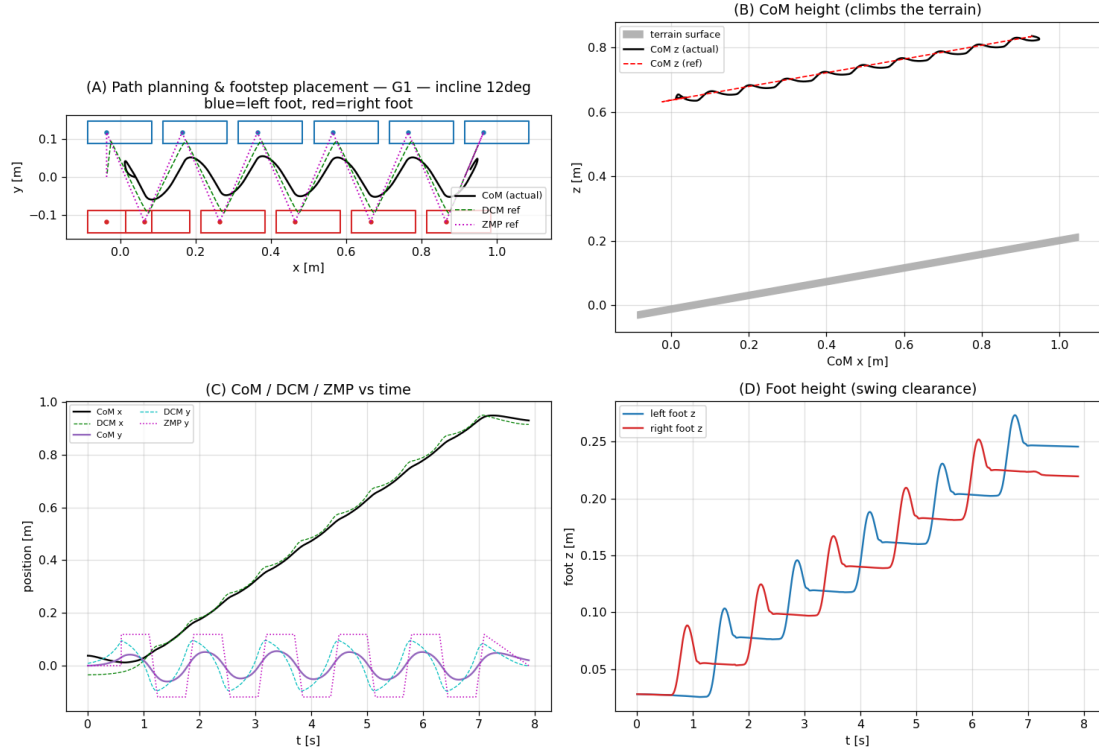


Figure 6: Walking up a 12° incline: (A) footstep placement, (B) CoM height climbing the slope, (C) CoM/DCM/ZMP, (D) swing-foot clearance.

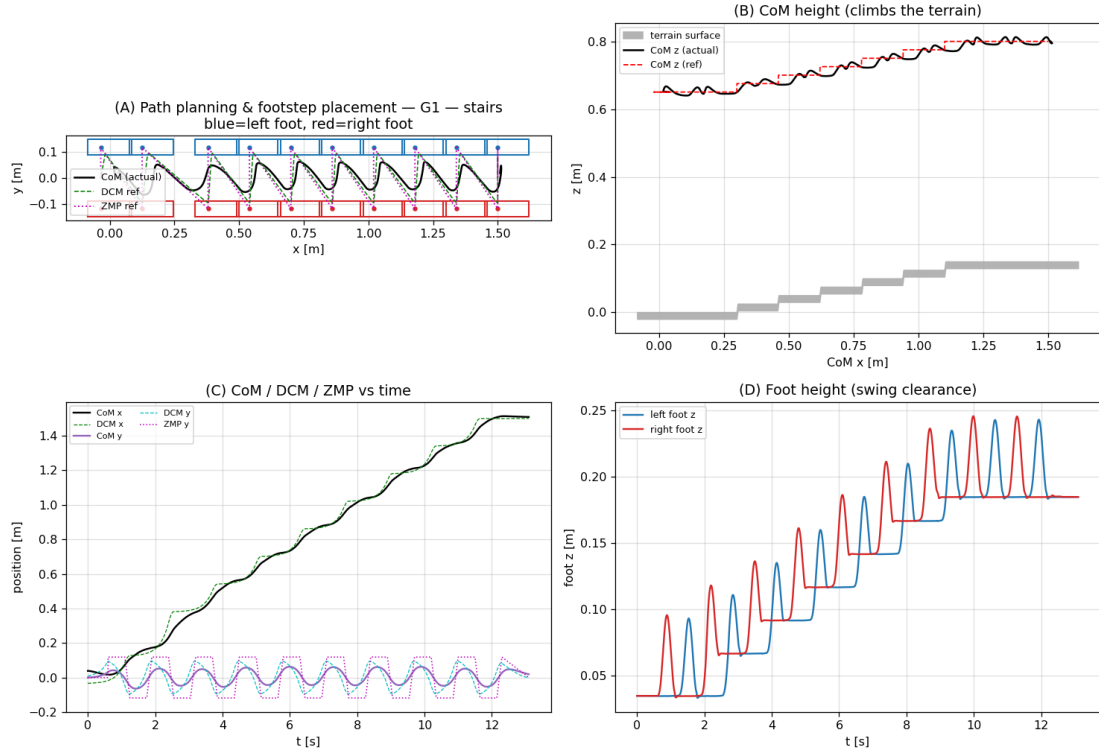


Figure 7: Climbing stairs (feet-together-per-tread): the CoM height (B) steps up with the treads; the swing foot (D) clears each riser.

5.5 Omnidirectional walking

The same DCM+QP stack walks in any direction from a body-frame velocity command (v_x, v_y, v_{yaw}) — forward, backward, strafing, and turning. The pelvis yaw tracks the heading, so the robot can walk a curve (Fig. 8).

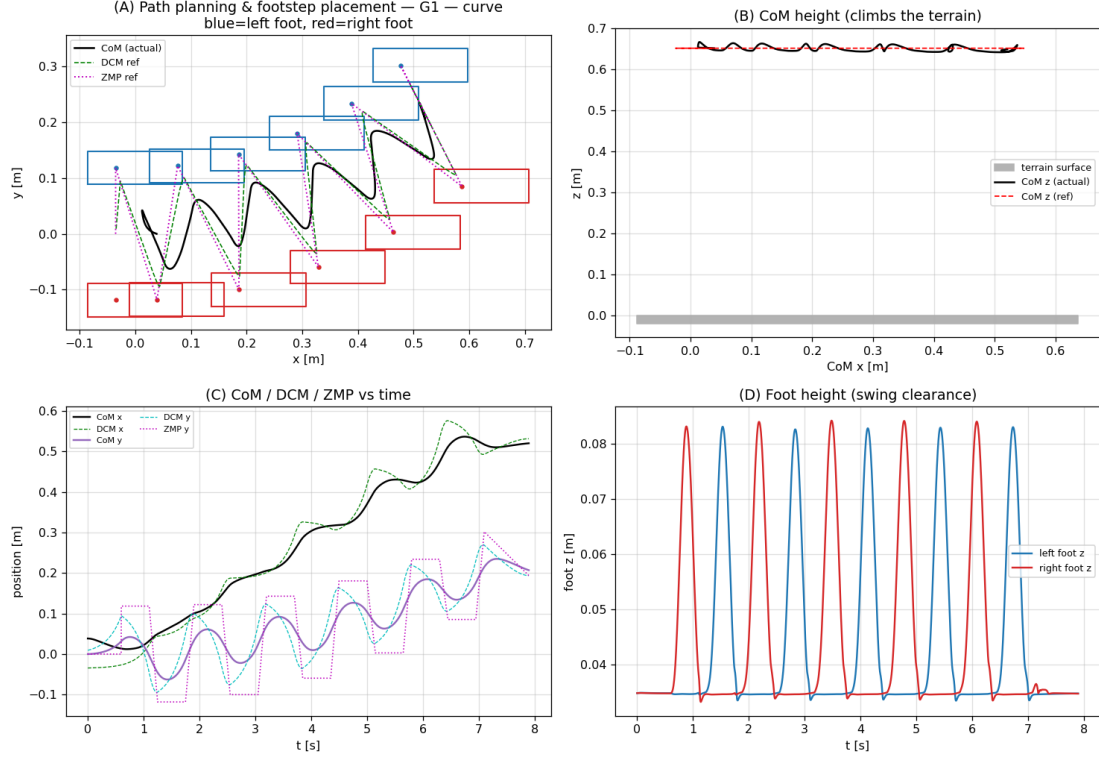


Figure 8: Walking a curve ($v_x = 0.10$, $v_{yaw} = 0.12$): the footstep rectangles rotate with the heading and the CoM follows a curved path.

5.6 Push recovery

While walking, external shoves are applied to the pelvis for ~ 100 ms. The capture-point step adjustment relocates the swing foot toward the perturbed DCM and the robot recovers without falling (Fig. 9); the tolerated impulse is larger fore-aft than laterally, reflecting the narrow foot.

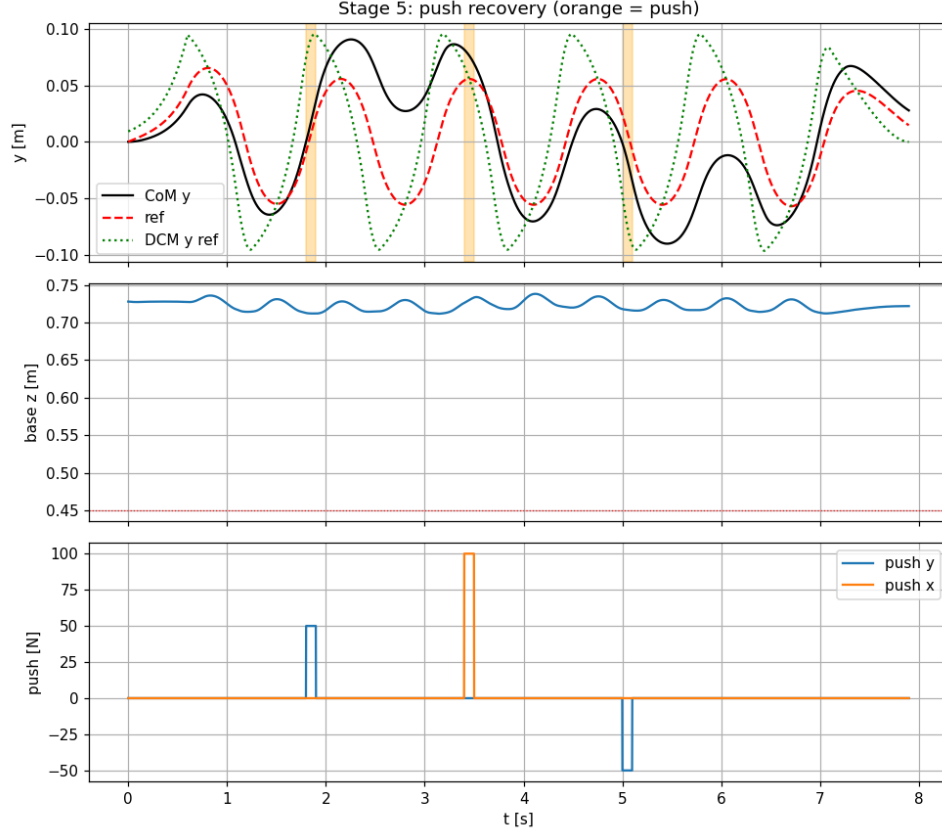


Figure 9: Push recovery during walking: lateral CoM vs. reference (pushes shaded), base height, and the applied force; the robot stays upright and re-converges.

5.7 Second robot: PAL Talos

The identical controller, planner and QP run on the 94 kg Talos through its `RobotConfig`; it stands, balances, walks on flat ground and inclines, and recovers from pushes (Fig. 10), demonstrating that the approach is not tuned to a single robot.

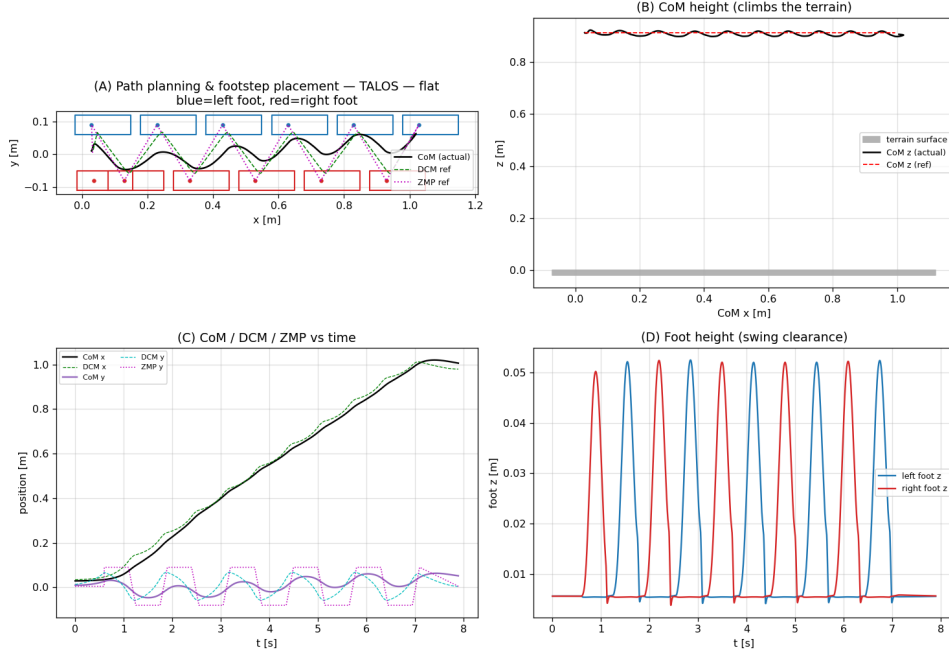


Figure 10: PAL Talos walking on flat ground with the same controller as the G1.

6 Incline Limit Analysis

How steep a slope can the robot hold? There are two distinct failure modes, each with a simple closed form.

Slipping. The tangential component of gravity must be held by friction: $mg \sin \alpha \leq \mu mg \cos \alpha$, i.e.

$$\alpha_{\text{slip}} = \arctan \mu. \quad (4)$$

With $\mu = 0.5$ this gives $\alpha_{\text{slip}} \approx 26.6^\circ$.

Tipping. The body tips about the downhill edge of the support when the gravity line passes outside it: $mg \cos \alpha d \geq mg \sin \alpha H$, i.e. $\alpha_{\text{tip}} = \arctan(d/H)$, where d is the horizontal distance from the CoM to the downhill edge and H the CoM height. Because the active controller keeps the CoM well inside the support polygon, d/H stays large and tipping never binds before slipping — so *standing is slip-limited*.

Experiment. Sweeping the slope angle, the robot stands stably up to 26° and slips at 27° , matching $\arctan \mu$ almost exactly. Note that uphill *walking* fails earlier ($\sim 16^\circ$): there the limit is dynamic (single-support balance and the unmodelled along-slope gravity in the horizontal LIPM), not static slip or tip.

7 Extended Demonstrations

Two integrated demos exercise the full stack (path planning $\rightarrow$ DCM $\rightarrow$ QP) on harder, less-scripted scenarios.

7.1 Waiter: slalom navigation around tables

The robot is given a goal behind a field of randomly generated round tables and must walk to it while carrying a tray with a frappe. The course is a *slalom*: tables are placed alternately left/right of the centre-line (one per gate), and the planned trajectory is built directly as a sinusoid in *opposite phase* to the tables, so the robot threads each gate on the free side (Fig. 11). The gate spacing is chosen so the sinusoid’s curvature stays within the gait’s turn rate, and a walkability check resamples the (random) layout until the path both clears every table and is gentle enough to follow. The footstep planner places steps along this path (pelvis facing the path tangent), the DCM+QP walk it, and the tray — rigidly attached to the upright torso — stays level so the frappe does not spill (measured torso tilt $<6^\circ$).

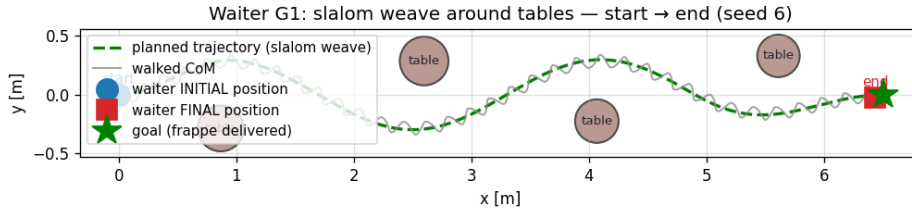


Figure 11: Waiter task: the planned slalom trajectory weaves between randomly generated tables; the walked CoM follows it from the initial to the final position, delivering the frappe at the goal.

7.2 Sisyphus: pushing a boulder up a slope

The robot reaches both arms forward and pushes a large ball (at hand height, constrained to slide along the slope) up an incline while walking uphill (Fig. 12). A steady forward CoM bias counters the backward-tipping torque from the high contact point — exactly as a person leans into a push. The ball advances in a push–rollback–push rhythm, climbing with the robot.

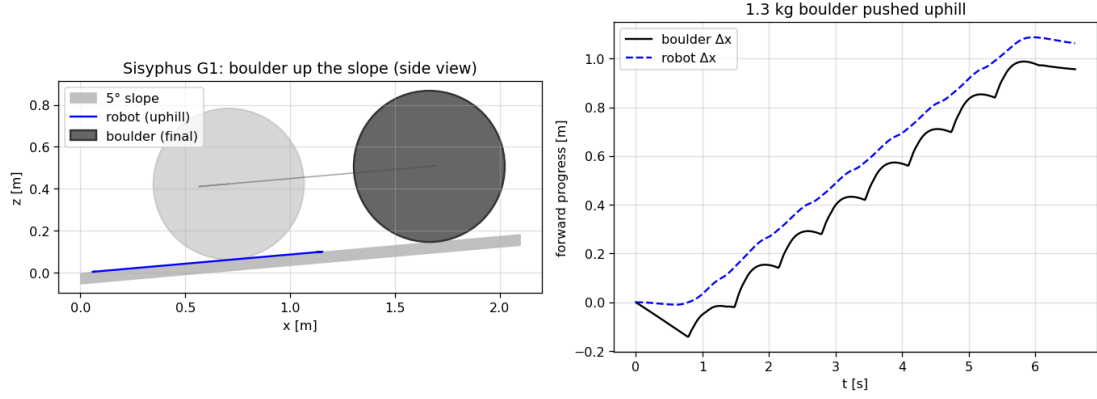


Figure 12: Sisyphus: (left) side view of the robot and the ball climbing the slope; (right) forward progress of the ball and the robot vs. time.

8 Parameters

All tunables live in `config/params.yaml`. The most important ones are summarised in Table 1.

Table 1: Key parameters (from `config/params.yaml`).

Parameter	Value	Meaning
<i>Whole-body QP</i>		
friction μ	0.5	friction coefficient (conservative)
$f_{\min}$	1 N	min. normal force per contact point
solver	quadprog	dense PD solver (clarabel/osqp fallback)
<i>Task weights / gains (w, K_p, K_d)</i>		
CoM	100, 100, 20	high-weight CoM task
orientation	20, 80, 18	torso upright vs. gravity
posture	1, 20, 8	regularisation toward the crouch
swing foot	120, 300, 34	airborne-foot tracking
k_{dcm}	3.0	DCM feedback gain
<i>Gait</i>		
step length	0.10–0.20 m	speed knob (≈ 0.11 – 0.23 m/s)
$t_{\text{SS}} / t_{\text{DS}}$	0.5 / 0.15 s	single- / double-support duration
swing apex	0.05 m	swing-foot peak height
<i>Capture point</i>		
gain / max shift	1.0 / 0.12 m	DCM-based step adjustment

9 Discussion, Limitations and Future Work

The dynamics-first, torque-level design meets all of the assignment’s core tasks: the robot walks continuously, plans and controls its CoM, shifts its weight onto the support foot, coordinates foot placement with body motion, stays stable through DS $\leftrightarrow$ SS transitions, and recovers from pushes and imperfect foot placement. Expressing balance with respect to gravity and the contact surface is what makes the controller generalise to inclines and stairs with no redesign, and the `RobotConfig` abstraction is what lets it run on a second robot.

Several limitations remain. (i) Uphill *walking* is capped near 16°: the planner uses a horizontal LIPM/DCM that ignores the along-slope gravity component; a slope-projected LIPM or an MPC formulation would push this toward the static slip limit. (ii) Lateral push tolerance is lower than fore–aft because of the narrow feet and the limited centre-of-pressure authority in single support; step-*timing* adaptation (landing early to catch a fall), in addition to the current

step-*placement* adjustment, would help. (iii) Stair climbing is tuned to gentle risers; taller steps need a larger step length and clearance. (iv) The QP backend is `quadprog` rather than `proxqp` (which has no Windows build) — the method is identical, only the solver differs. Natural next steps are a receding-horizon DCM/CoP MPC, an angular-momentum task for stronger push recovery, and online navigation that re-plans the velocity command continuously.

10 Conclusion

We built a torque-level, whole-body-QP walking controller for the Unitree G1 and PAL Talos in MuJoCo, organised as an offline (footsteps + ZMP/DCM) $\rightarrow$ online (DCM feedback + capture-point) $\rightarrow$ whole-body-control pipeline. The robot achieves stable continuous walking, single-support balancing, weight shifting, push recovery, omnidirectional walking, and locomotion over inclines and small stairs, with a simple slip-vs-tip analysis that correctly predicts the standing limit. Because every balance quantity is expressed with respect to gravity and the local surface, and because the robot-specific facts are isolated in one configuration object, the same code runs unchanged on both robots and extends naturally to the two integrated path-planning demonstrations.

References

- [1] Lecture 9, *Whole-Body Control via QP*, Robotic Systems I.
- [2] Lecture 10, *ZMP / LIPM / DCM and walking*, Robotic Systems I.
- [3] S. Kuindersma et al., *Optimization-based locomotion planning, estimation, and control design for Atlas*, Auton. Robots, 2016.
- [4] J. Engelsberger et al., *Three-dimensional bipedal walking control based on Divergent Component of Motion*, IEEE T-RO, 2015.
- [5] J. Pratt et al., *Capture Point: A step toward humanoid push recovery*, Humanoids, 2006.